SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
Program Name: B. Tech	**Assignment Type: Lab**	**Academic Year:**2026-2027

Q.No.	Question	*Expected Time to complete*

1	**Lab 13: Code Refactoring – Improving Legacy Code with AI Suggestions**
	Lab Objectives:
	• Identify code smells and inefficiencies in legacy Python scripts.
	• Use AI-assisted coding tools to **refactor** for readability, maintainability, and performance.
	• Apply **modern Python best practices** while ensuring output correctness.
	--- 	
	Task Description #1 (Refactoring – Removing Code Duplication)
	• Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
	• Instructions:
	o Prompt AI to identify duplicate logic and replace it with functions or classes.
	o Ensure the refactored code maintains the same output.
	o Add docstrings to all functions.
	• Sample Legacy Code:
	# Legacy script with repeated logic
	print("Area of Rectangle:", 5 * 10)
	print("Perimeter of Rectangle:", 2 * (5 + 10))
	
	print("Area of Rectangle:", 7 * 12)
	print("Perimeter of Rectangle:", 2 * (7 + 12))
	
	print("Area of Rectangle:", 10 * 15)
	print("Perimeter of Rectangle:", 2 * (10 + 15))
	• Expected Output:
	o Refactored code with a reusable function and no
		Week7 - Monday

duplication.

Task Description #2 (Refactoring – Optimizing Loops and Conditionals)

- Task: Use AI to analyze a Python script with nested loops and complex conditionals.
- Instructions:
  - Ask AI to suggest algorithmic improvements (e.g., replace nested loops with set lookups or comprehensions).
  - Implement changes while keeping logic intact.
  - Compare execution time before and after refactoring.
- Sample Legacy Code:

```
# Legacy inefficient code
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

- Expected Output:
  - Optimized code using set lookups with performance comparison table.

Task Description #3 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
  - Identify repeated or related logic and extract it into reusable functions.
  - Ensure the refactored code is modular, easy to read, and documented with docstrings.

- Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
  - Code with a function calculate_total(price) that can be reused for multiple price inputs.

Task Description #4 (Refactoring – Replacing Hardcoded Values with Constants)

- Task: Use AI to identify and replace all hardcoded "magic numbers" in the code with named constants.
- Instructions:
  - Create constants at the top of the file.
  - Replace all hardcoded occurrences in calculations with these constants.
  - Ensure the code remains functional and is easier to maintain.
- Sample Legacy Code:

```
# Legacy script with hardcoded values
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

- Expected Output:
  - Code with constants like PI = 3.14159 and RADIUS = 7 used in calculations.

Task Description #5 (Refactoring – Improving Variable Naming and Readability)

- Task: Use AI to improve readability by renaming unclear variables and adding inline comments.
- Instructions:
  - Replace vague names with meaningful ones.
  - Add comments where logic is not obvious.

- Keep functionality exactly the same.

- Sample Legacy Code:

```
# Legacy script with poor variable names
a = 10
b = 20
c = a * b / 2
print(c)
```

- Expected Output:
  - Code with descriptive variable names like base, height, area_of_triangle, plus explanatory comments.

Task Description #6 (Refactoring – Removing Redundant Conditional Logic)

• Task: Use AI to refactor a Python script that contains repeated if–else logic for grading students.

• Instructions:

o Ask AI to identify redundant conditional checks.

o Replace them with a reusable function.

o Ensure grading logic remains unchanged.

• Code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")


marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
```

	print("Grade B") else: print("Grade C") • Expected Output: o A reusable function like calculate_grade(marks) with clean logic and docstring.	
	Task Description #7 (Refactoring – Converting Procedural Code to Functions) **• Task: Use AI to refactor procedural input–processing logic into functions.** • Instructions: o Identify input, processing, and output sections. o Convert each into a separate function. o Improve code readability without changing behavior. • Sample Legacy Code: num = int(input("Enter number: ")) square = num * num print("Square:", square) • Expected Output: o Modular code using functions like get_input(), calculate_square(), and display_result().	
	Task Description #8 (Refactoring – Optimizing List Processing) **• Task: Use AI to refactor inefficient list processing logic.** • Instructions: o Ask AI to replace manual loops with list comprehensions or built-in functions.	

	o Ensure output remains identical. • Sample Legacy Code: nums = [1, 2, 3, 4, 5] squares = [] for n in nums: squares.append(n * n) print(squares) • Expected Output: o Optimized version using list comprehension with improved readability	